



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science

Faculty of Computing

Project

Programme : Bachelor of Computer Science (*Data Engineering*)

Subject Code : SECR2043

Subject Name : Operating System

Session-Sem : 2024/2025-2

Prepared by : Cheryl Cheong Kah Voon (A23CS0060)

Lau Yee Wen (A23CS0099)

Poh Lok Yee (A23CS0262)

Woo Cheng Shuan (A23CS0283)

Section : 02

Lecturer : DR. MUHAMMAD ZAFRAN BIN MUHAMMAD ZALY
SHAH

Date : 06-07-2025

Table of Content

1.0 Title & Abstract	4
1.1 Project Title.....	4
1.2 Abstract	4
2.0 Introduction	5
2.1 Background of CPU Scheduling in Operating Systems	5
2.2 Overview of Non-Preemptive Scheduling	5
2.3 Importance of SJF and Priority Scheduling in Real-World Systems.....	5
3.0 Problem Statement	7
3.1 Motivation	7
3.2 Real-World Challenge in Patient Scheduling	7
3.3 Objectives of the Project	7
4.0 Related Works	8
4.1 Existing Scheduling Simulators.....	8
4.2 Research on Non-Preemptive Algorithms.....	8
4.3 Studies on Hospital Queue Optimization.....	9
5.0 Methodology	10
5.1 Mapping Real-World Scenario to OS Concepts.....	10
5.2 Patient Data Structure and Attributes.....	10
5.3 Scheduling Logic	10
5.4 Tools and Technologies Used	11
6.0 Implementation	12
6.1 System Overview	12
6.2 System Flowchart	13
6.3 Code Implementation.....	13
7.0 Discussion	16
7.1 Test Cases and Patient Inputs	16
7.2 Gantt Charts for SJF and Priority Scheduling	16
7.3 Performance Metrics (Waiting Time, Turnaround Time).....	16
7.4 Comparative Analysis and Observations	17
8.0 Future Work & Conclusion	18

8.1 Limitations of Current Implementation.....	18
8.2 Potential Enhancements	18
8.3 Summary of Findings	18
Acknowledgement	19
References	20
Appendices	21
Test Outputs	21
YouTube Video Link.....	21

1.0 Title & Abstract

1.1 Project Title

Simulation of Non-Preemptive Scheduling Algorithms (SJF and Priority Scheduling) in a Hospital Patient Management System

1.2 Abstract

Process scheduling in operating systems is essential for establishing the sequence in which processes inside a system execute. Shortes Job First (SJF) and Priority Scheduling are two non-preemptive CPU scheduling algorithms that are investigated and simulated in this study by using a real-world healthcare scenario: hospital patient management.

Patients are represented as processes, treatment durations as CPU burst times and condition severity as priority levels in the suggested simulation. As the CPU, the doctor schedules patients according to their highest priority (Priority Scheduling) or shortest treatment duration (SJF). A better comprehension of scheduling concepts and how they affect performance indicators like turnaround and waiting times is made possible by this analogy.

A visual depiction of the scheduling process is provided by the simulation, which is built using C++. To compare the two algorithms' efficacy, Gantt charts and output summaries are produced. The findings show how overall efficiency and patient service experience are impacted by non-preemptive scheduling decisions. This project intends to give students learning operating system fundamentals both educational insight and practical application.

2.0 Introduction

2.1 Background of CPU Scheduling in Operating Systems

The CPU is one of the many system resources that an operating system (OS) is in charge of controlling. The OS must use a technique known as CPU scheduling to decide which process gets access first because numerous programs frequently request CPU access at the same time. System responsiveness, throughput and resource usage are all improved by effective scheduling.

In general, scheduling algorithms can be divided into two groups: preemptive and non-preemptive. When using non-preemptive scheduling, a task must finish using the CPU before another process may be scheduled. Shortest Job First (SJF) and Priority Scheduling are two popular non-preemptive scheduling algorithms that seek to maximize performance by cutting down on waiting and turnaround times.

2.2 Overview of Non-Preemptive Scheduling

The process with the least expected CPU burst time is chosen by SJF. Although this approach reduces the average waiting time, it necessitates knowing the burst time of each operation beforehand, which isn't always possible in dynamic systems.

In contrast, Priority Scheduling gives each process a priority value. The process chosen for execution is the one with the highest priority, which is usually indicated by the lowest numerical value. This approach works well in systems where tasks need to be ranked in order of priority, urgency or user role.

Both SJF and Priority Scheduling in non-preemptive mode enable a task to finish without interruption once it starts execution, in contrast to preemptive algorithms.

2.3 Importance of SJF and Priority Scheduling in Real-World Systems

Knowing how scheduling algorithms work is essential for real-world service systems like manufacturing lines, contact centres and hospitals as well as for computing environments. We simulate a hospital patient management system in this project, where patients come at different times, receive different treatments and have different urgency (priority) levels.

By simulating patient flow using SJF and Priority Scheduling, the project demonstrates how different scheduling strategies affect:

- Patient waiting time
- Service order
- Resource optimization

This real-world analogy helps students and practitioners grasp OS scheduling logic more intuitively by translating abstract scheduling concepts into familiar, human-centered situations like patient treatment.

3.0 Problem Statement

3.1 Motivation

In health care hospital emergency areas, patient scheduling is very crucial to save lives as well as using medical resource maximally. The problem with medical staff is that they sometimes must attend to several patients who come at different times and at different urgencies. However, scheduling these patients efficiently, fairly, and accurately remains a challenge. By drawing an analogy between hospital workflows and CPU scheduling, we aim to enhance understanding of how operating systems prioritize tasks through scheduling algorithms. This project takes the scenario of hospital patient treatment by using it as an analogy to explain non-preemptive scheduling concepts, as such, it is easier to introduce students to abstract OS concepts to them and to put into their mind the applicability of such algorithms in more general non-computing contexts.

3.2 Real-World Challenge in Patient Scheduling

In a hospital, patients arrive at different times, require varying treatment durations, and have different levels of urgency. This is like the procedure in an operating system that has arrival time, burst time and priorities. Using only First-Come, First-Served (FCFS) can delay treatment for critical patients, while focusing solely on urgency may lead to long waits for others. The balance is required to avoid unfairness and wastage. Priority Scheduling and Shortest Job First (SJF) fall in the category of non-preemptive scheduling because the moment a treatment is initiated, it cannot be stopped. This makes it an appropriate and learning model to simulate patient scheduling.

3.3 Objectives of the Project

The main objective of this project is to simulate a real-world hospital patient management system using non-preemptive scheduling algorithms, specifically Shortest Job First (SJF) and Priority Scheduling. By applying these algorithms, the project aims to demonstrate how scheduling decisions affect key performance metrics such as waiting time and turnaround time. To enhance understanding, the simulation includes visual representations like Gantt charts, allowing users to clearly see the sequence and timing of treatments. This project also serves to bridge the gap between theoretical operating system concepts and practical, real-life applications. Ultimately, it is designed as an educational tool to help students better grasp the logic and impact of different scheduling strategies in both computing and real-world scenarios.

4.0 Related Works

4.1 Existing Scheduling Simulators

Simulators are important in guiding students and researchers the functional mechanisms of CPU scheduling algorithms. These simulators are particularly effective in the academic context since they enable students to experiment with various algorithms interactively and see how to schedule in real-time.

Diaz, Batista and Castro (2007), presented a real-time scheduling simulator, known as Realtss, in order to support both real-time scheduling and dynamic scheduling algorithms. This tool enables the user to set scheduling parameters, graphical performance analysis and graphic representation of timing execution. Although it primarily focuses on real-time environments, the simulator reinforces how critical scheduling is for system responsiveness and serves as a foundation for educational use in general OS instruction [1].

Likewise, Buck and Perugini (2018) have presented the educational interactive tool that enables the student to simulate the operation of several scheduling algorithms CPU, including the FCFS, SJF, Round Robin, and Priority Scheduling algorithms, and to automatically create a Gantt chart. Goel and Garg (2013) also underline the applicability of the comparison of algorithms in terms of simulation in the academic context, as the tools contribute to disambiguating the trade-off defined between efficiency, fairness, and complexity [2][3].

While these simulators provide valuable technical insights, many lack real-world analogies that help learners intuitively grasp how scheduling decisions affect practical systems, such as hospitals or customer service operations. This gap is what our project aims to fill by applying scheduling algorithms in a hospital patient treatment simulation.

4.2 Research on Non-Preemptive Algorithms

Non-preemptive scheduling algorithms, such as Shortest Job First (SJF) and Priority Scheduling, have been widely studied due to their predictability and simplicity in environments where context switching is not preferred. SJF is known to reduce average waiting time because the shortest available task is chosen but this technique requires the complete knowledge of burst times which cannot be possible in all cases. P In contrast, priority scheduling chooses tasks on a priority or urgency basis which gives it the suitability in such a scenario as emergency healthcare systems Nevertheless, a well-known drawback of Priority Scheduling is starvation, which occurs when a process with a low priority is delayed indefinitely. To address this, techniques like aging are often used to gradually increase a task's priority over time.

According to Pemasinghe and Rajapaksha (2022), the article thoroughly compared several CPU scheduling algorithms, FCFS, SJF, Priority-Based, and Multilevel Queuing. They confirm

their research on the fact that SJF works best in the situations it is possible to predict the burst time and that Priority Scheduling could be flexible in such aspects that it needs to be balanced, otherwise, it can lead to the case of starvation [4]. Lin et al. (2017) similarly applied non-preemptive rules in a healthcare setting and observed reduced wait times when patient urgency was considered [5]. Goel and Garg (2013) also emphasized that while SJF optimizes performance in theory, fairness can sometimes be sacrificed [3]. These studies collectively reinforce the importance and practicality of non-preemptive algorithms in systems where predictability and urgency are key considerations.

4.3 Studies on Hospital Queue Optimization

In hospital environments, especially in emergency and outpatient departments, optimizing patient queues is crucial to improving service quality and reducing waiting times. Patient flow and various types of scheduling can be modeled using discrete-event simulation (DES) in order to test the strategies. For example, Comas et al. (2008) proved that the efficiency of planning cataract surgery using priority-based approach minimized delays and enhanced resource management [6]. Similarly, Chen et al. (2022) used simulation-optimization to schedule ultrasound appointments, demonstrating that the resulting efficiency and reduced wait times are achieved by adjusting the room assignment to the urgency of the patient [7].

These studies confirm the use of SJF and Priority Scheduling principles, widespread in the OS theory, in the field of healthcare. By incorporating urgency and treatment duration into scheduling decisions, both research and practice show meaningful improvements in patient outcomes. This supports our project's use of hospital-based simulation as an effective educational tool for illustrating non-preemptive scheduling.

5.0 Methodology

This project simulates and compares two classic non-preemptive CPU scheduling algorithms which are Shortest Job First and Priority Scheduling. By applying them to a real-world hospital emergency room scenario, it helps to understand how different scheduling decisions affect patient treatment order, waiting time, and overall system efficiency.

5.1 Mapping Real-World Scenario to OS Concepts

This simulation maps the roles in a hospital emergency department to elements of CPU scheduling in operating systems. The hospital simulation represents:

Hospital Context	OS Concept Equivalent
Patient	Process
Treatment Time	CPU Burst Time
Patient Arrival Time	Process Arrival Time
Condition Severity	Process Priority
Doctor	CPU

This mapping provides a relatable way to understand how operating systems manage competing tasks.

5.2 Patient Data Structure and Attributes

Each patient is represented as a structured object containing relevant scheduling and clinical data. The selected attributes serve both logical and algorithmic purposes.

Attribute	Description
id	Unique patient identifier (e.g. P1, P2)
arrival	Time when the patient enters the system
burst	Duration of treatment
priority	Urgency level
start	Time when treatment start
finish	Time when treatment end
waiting	Waiting time before treatment
turnaround	Total time from arrival to completion
done	Boolean flag to mark if the patient has been treated

5.3 Scheduling Logic

1. Shortest Job First

Patients are treated in order of the shortest treatment time among those who have arrived. If multiple patients arrive at the same time, the one with the shortest burst time is selected first.

2. Priority Scheduling

Patients are treated in order of urgency which the lower the priority number, the more critical the case. If multiple patients share the same priority, the one who arrived first is treated first.

Each scheduling algorithm is implemented independently using the same patient dataset. After simulation, key performance metrics are calculated and compared.

5.4 Tools and Technologies Used

This project was implemented using the C++ programming language. C++ was chosen because it provides precise control over memory, data structure and logic flow. No external libraries or graphical interfaces were used.

6.0 Implementation

This section describes how each algorithm was implemented using C++, the system logic, and how metrics like waiting and turnaround time are calculated for analysis.

6.1 System Overview

Each patient is treated as a process, and their attributes which are arrival time, burst time (treatment duration), and priority (condition severity) are used to determine treatment order. The system mimics the behaviour of a non-preemptive CPU, where treatment runs to completion without switching.

Metric Calculation

For each patient, the following metrics are calculated during simulation:

- Start Time = When Treatment begins
- Finish Time = Start Time + Burst Time
- Waiting Time = Start Time – Arrival Time
- Turnaround Time = Finish Time – Arrival Time

At the end, the system computes:

- Average Waiting Time: Average how long a patient waited before treatment started.
- Average Turnaround Time: Average total time from arrival to treatment completion.

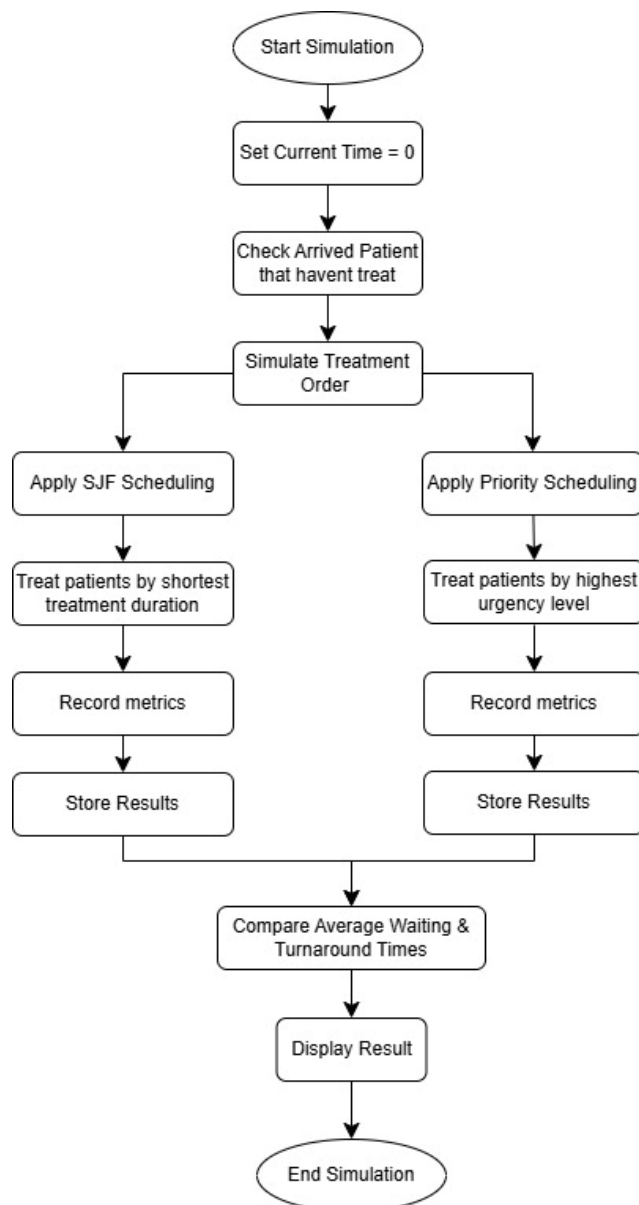
Shared Simulation Flow

The following steps are common to both algorithms:

1. Begin simulation at time 0.
2. Check which patients have arrived and are not yet treated.
3. From those available, select a patient based on the algorithm's rule:
 - **Shortest Job First**
select patient with shortest burst time
 - **Priority Scheduling**
select patient with highest priority
4. Treat the patient fully without interrupt.
5. Perform metric calculation.
6. Advance time to treatment finish.
7. Repeat until all patients are treated.

Once both algorithms complete their simulation, the system will print out each patients detailed metrics and compares the average waiting and turnaround times between SJF and Priority scheduling. This will show the different scheduling decisions affect overall hospital performance and patient satisfaction.

6.2 System Flowchart



6.3 Code Implementation

```
project.cpp
1  #include <iostream>
2  #include <vector>
3  #include <iomanip>
4  #include <algorithm>
5  using namespace std;
6
7  // Structure to store patient information
8  struct Patient {
9      string id;
10     int arrival, burst, priority, start = 0, finish = 0, waiting = 0, turnaround = 0;
11     bool done = false;
12 };
13
14 // Reset all timing values for reusing same data
15 void reset(vector<Patient> &patients) {
16     for (auto &p : patients) {
17         p.start = p.finish = p.waiting = p.turnaround = 0;
18         p.done = false;
19     }
20 }
```

```

// Run SJF Scheduling
void runSJF(vector<Patient> patients) {
    cout << "\n== Shortest Job First (SJF) Scheduling ==\n";
    int time = 0;
    int completed = 0;
    int total = patients.size();

    while (completed < total) {
        int idx = -1;
        int shortest = 9999;

        // Find shortest burst time among arrived patients
        for (int i = 0; i < total; i++) {
            if (!patients[i].done && patients[i].arrival <= time && patients[i].burst < shortest) {
                shortest = patients[i].burst;
                idx = i;
            }
        }

        if (idx == -1) {
            time++; // If no one has arrived, advance time
            continue;
        }

        // Process selected patient
        patients[idx].start = time;
        patients[idx].finish = time + patients[idx].burst;
        patients[idx].waiting = patients[idx].start - patients[idx].arrival;
        patients[idx].turnaround = patients[idx].finish - patients[idx].arrival;
        patients[idx].done = true;

        time = patients[idx].finish;
        completed++;
    }

    // Output
    float totalWait = 0, totalTurn = 0;
    cout << "ID\tArrival\tBurst\tStart\tFinish\tWaiting\tTurnaround\n";
    for (auto p : patients) {
        cout << p.id << "\t" << p.arrival << "\t" << p.burst << "\t" << p.start << "\t"
            << p.finish << "\t" << p.waiting << "\t" << p.turnaround << "\n";
        totalWait += p.waiting;
        totalTurn += p.turnaround;
    }
    cout << fixed << setprecision(2);
    cout << "Average Waiting Time: " << totalWait / total << endl;
    cout << "Average Turnaround Time: " << totalTurn / total << endl;
}

```

```

// Run Priority Scheduling
void runPriority(vector<Patient> patients) {
    cout << "\n=== Priority Scheduling ===\n";
    int time = 0;
    int completed = 0;
    int total = patients.size();

    while (completed < total) {
        int idx = -1;
        int bestPriority = 9999;

        // Find highest priority (lowest number) among arrived patients
        for (int i = 0; i < total; i++) {
            if (!patients[i].done && patients[i].arrival <= time) {
                if (patients[i].priority < bestPriority) {
                    bestPriority = patients[i].priority;
                    idx = i;
                } else if (patients[i].priority == bestPriority) {
                    // If same priority, pick the earlier arrival
                    if (patients[i].arrival < patients[idx].arrival)
                        idx = i;
                }
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }

        // Process selected patient
        patients[idx].start = time;
        patients[idx].finish = time + patients[idx].burst;
        patients[idx].waiting = patients[idx].start - patients[idx].arrival;
        patients[idx].turnaround = patients[idx].finish - patients[idx].arrival;
        patients[idx].done = true;

        time = patients[idx].finish;
        completed++;
    }

    // Output
    float totalWait = 0, totalTurn = 0;
    cout << "ID\tArrival\tBurst\tPriority\tStart\tFinish\tWaiting\tTurnaround\n";
    for (auto p : patients) {
        cout << p.id << "\t" << p.arrival << "\t" << p.burst << "\t" << p.priority << "\t\t"
            << p.start << "\t" << p.finish << "\t" << p.waiting << "\t" << p.turnaround << "\n";
        totalWait += p.waiting;
        totalTurn += p.turnaround;
    }
    cout << fixed << setprecision(2);
    cout << "Average Waiting Time: " << totalWait / total << endl;
    cout << "Average Turnaround Time: " << totalTurn / total << endl;
}

int main() {
    // Patient list (id, arrival time, burst time, priority)
    vector<Patient> patients = {
        {"P1", 0, 5, 3},
        {"P2", 1, 8, 1},
        {"P3", 2, 6, 2},
        {"P4", 3, 7, 4}
    };

    // Run SJF
    runSJF(patients);

    // Reset and run Priority
    reset(patients);
    runPriority(patients);

    return 0;
}

```

7.0 Discussion

7.1 Test Cases and Patient Inputs

To evaluate the effectiveness of the non-preemptive scheduling algorithm, a test scenario was created based on a realistic hospital setting. Each “process” represents a patient requiring treatment and their attributes correspond to:

- Arrival Time: When the patient enters the emergency room
- Burst Time: Estimated treatment duration
- Priority: Urgency level (lower value means more critical condition)

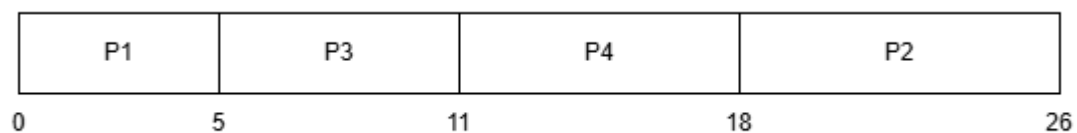
Test Data:

Patient	Arrival Time	Burst Time	Priority
P1	0	5	3
P2	1	8	1
P3	2	6	2
P4	3	7	4

7.2 Gantt Charts for SJF and Priority Scheduling

Shortest-Job-First (SJF)

Order: P1 -> P3 -> P4 ->P2



Priority Scheduling

Order: P1 -> P2 -> P3 ->P4



7.3 Performance Metrics (Waiting Time, Turnaround Time)

Shortest-Job-First (SJF)

ID	Waiting Time	Turnaround Time
----	--------------	-----------------

P1	0	5
P2	17	25
P3	3	9
P4	8	15

Average Waiting Time: $(0+17+3+8)/4 = 7s$

Average Turnaround Time: $(5+25+9+15)/4 = 13.5s$

Priority Scheduling

ID	Waiting Time	Turnaround Time
P1	0	5
P2	4	12
P3	11	17
P4	16	23

- Although P2 has the highest priority, P1 was treated first because it arrived earlier, and the system uses non-preemptive scheduling
- Since P2 hadn't arrive at time 0, it could not preempt P1 once treatment began.

Average Waiting Time: $(0+4+11+16)/4 = 7.75s$

Average Turnaround Time: $(5+12+17+23)/4 = 14.25s$

7.4 Comparative Analysis and Observations

This comparison between SJF and Priority Scheduling are shown below:

- SJF has shorter average waiting time (7s) and turnaround time (13.50s), which improves overall system efficiency.
- However, SJF may delay the critical patient, which is P2. In the stimulation, P2 has the highest priority but was treated at last in SJF, since it has the longest treatment time.
- SJF may lead to starvation for patients with long treatment time if new short duration cases keep arriving.
- Priority Scheduling has the longer waiting time (7.75s) and turnaround time (14.25s).
- Priority Scheduling reflects the real-world medical ethics by ensuring that the most critical patients are treated first, even if they arrive later or need more time.
- Even though P2 has the highest priority, it is not treated first because it only arrived after P1 had already started treatment. This highlights on how non-preemptive scheduling respects arrival time now of decision.
- Overall, the analysis confirms that while SJF improves performance metrics, it compromises fairness in life critical scenario. Priority Scheduling sacrifices some efficiency some efficiency but aligns with real-world ethical expectations. This comparison demonstrates how scheduling decisions directly impact not only system performance but also human outcomes, especially in healthcare environments.

8.0 Future Work & Conclusion

8.1 Limitations of Current Implementation

The current simulation uses a static patient list and assumes all data is known in advance. There is no real-time arrival stimulation, dynamic priority adjustment or user interface. The model also does not support preemptive scheduling, which could better handle patients whose conditions suddenly worsen. Additionally, the program runs entirely in the console with no visual feedback, which may limit usability.

8.2 Potential Enhancements

To improve the system's realism and usability, several features can be added:

- Real-time patients input and arrival simulation using dynamic queues
- Priority escalation, if the patient's condition worsens
- Preemptive algorithm support such as preemptive priority scheduling

This improvement would make the simulator more realistic

8.3 Summary of Findings

The project demonstrates the application of non-preemptive CPU scheduling algorithms, Shortest Jobs First (SJF) and Priority Scheduling in a hospital environment. The simulation uses patient arrival time, treatment duration and condition severity as scheduling criteria.

The overall results show that SJF minimizes average wait time and turnaround time but may delay urgent patients. Priority Scheduling better mirrors hospital practices by ensuring critical patients are treated first.

While SJF is more efficient computationally, Priority Scheduling provides a more humane and ethically aligned solution. This project highlights the importance of selecting scheduling strategies that balance performance with fairness, especially in real-world systems where decisions affect people's lives.

This project shows that operating system algorithms are not just theoretical. When applied to real-life scenarios like hospital emergencies, they carry human impact. Choosing the right scheduling strategy is not only about performance, but also about fairness, urgency and ethical decision making.

Acknowledgement

We would like to express our sincere gratitude to our lecturer, Dr Muhammad Zafran Bin Muhammad Zaly Shah, for this continuous guidance, support and valuable feedback throughout the completion of this group project. His expertise in operating system concepts greatly enhanced our understanding of CPU scheduling and helped us apply it effectively in a real-world healthcare simulation. We also wish to thank our group members for their teamwork, commitment and effort in carrying out the research, simulation coding and preparation of the final report and presentation. Lastly, we appreciate the various academic and online resources that contributed to our understanding of non-preemptive scheduling algorithm and their applications in system design.

References

- [1] Diaz, A., Batista, R., & Castro, O. (2007). *Realtss: a real-time scheduling simulator*.
<https://doi.org/10.1109/icccc.2007.4344998>
- [2] Buck, J. W., & Perugini, S. (2018). An Interactive, Graphical CPU Scheduling Simulator for Teaching Operating Systems. Retrieved June 26, 2025, from arXiv.org website:
<https://arxiv.org/abs/1812.05160>
- [3] Goel, N., & Garg, R. B. (2013). A Comparative Study of CPU Scheduling Algorithms. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1307.4165>
- [4] Sajeewa Pemasinghe, & Rajapaksha, S. (2022). *Comparison of CPU Scheduling Algorithms: FCFS, SJF, SRTF, Round Robin, Priority Based, and Multilevel Queuing*.
<https://doi.org/10.1109/r10-htc54060.2022.9929533>
- [5] Lin, C. K. Y., Ling, T. W. C., & Yeung, W. K. (2017). Resource Allocation and Outpatient Appointment Scheduling Using Simulation Optimization. *Journal of Healthcare Engineering, 2017*, 1–19. <https://doi.org/10.1155/2017/9034737>
- [6] Comas, M., Castells, X., Hoffmeister, L., Román, R., Cots, F., Mar, J., ... Espallargues, M. (2008). Discrete-Event Simulation Applied to Analysis of Waiting Lists. Evaluation of a Prioritization System for Cataract Surgery. *Value in Health, 11*(7), 1203–1213. <https://doi.org/10.1111/j.1524-4733.2008.00322.x>
- [7] Chen, P.-S., Gary Yu-Hsin Chen, Liu, L.-W., Zheng, C.-P., & Huang, W.-T. (2022). Using Simulation Optimization to Solve Patient Appointment Scheduling and Examination Room Assignment Problems for Patients Undergoing Ultrasound Examination. *Healthcare (Basel), 10*(1), 164–164. <https://doi.org/10.3390/healthcare10010164>

Appendices

Test Outputs

```
C:\Users\Polok\OneDrive - Universiti Teknologi Malaysia (UTM)\Documents\Y2S2\OS\project.exe

=== Shortest Job First (SJF) Scheduling ===
ID    Arrival Burst   Start   Finish   Waiting Turnaround
P1     0         5        0        5        0         5
P2     1         8       18       26       17       25
P3     2         6        5       11        3         9
P4     3         7       11       18        8       15
Average Waiting Time: 7.00
Average Turnaround Time: 13.50

=== Priority Scheduling ===
ID    Arrival Burst   Priority   Start   Finish   Waiting Turnaround
P1     0         5         3        0         5        0         5
P2     1         8         1        5       13        4        12
P3     2         6         2       13       19       11        17
P4     3         7         4       19       26       16        23
Average Waiting Time: 7.75
Average Turnaround Time: 14.25

-----
Process exited after 0.1894 seconds with return value 0
Press any key to continue . . .
```

YouTube Video Link

https://youtu.be/9WBfke4J7_k